Introduction to JavaScript/Node.js

JavaScript is a Programming Language

- Just like any other programming language, JavaScript has:
  - Loops
  - Variables (dynamically typed)
  - Functions/Methods
  - etc.

Node.js is a Runtime

- All browsers have JavaScript engines that allow JavaScript to be run on the “client”.
- Safari has JavaScriptCore
- Firefox has Spidermonkey
- Chrome uses “V8” - which is what Node.js uses

Node.js (cont.)

- Node.js is simply a runtime built on top of the V8 engine (not to be confused with the car)
- This allows JavaScript to be taken out of the browser (client-side) and be used elsewhere (server-side).
- So JavaScript is the language, Node.js is the runtime.

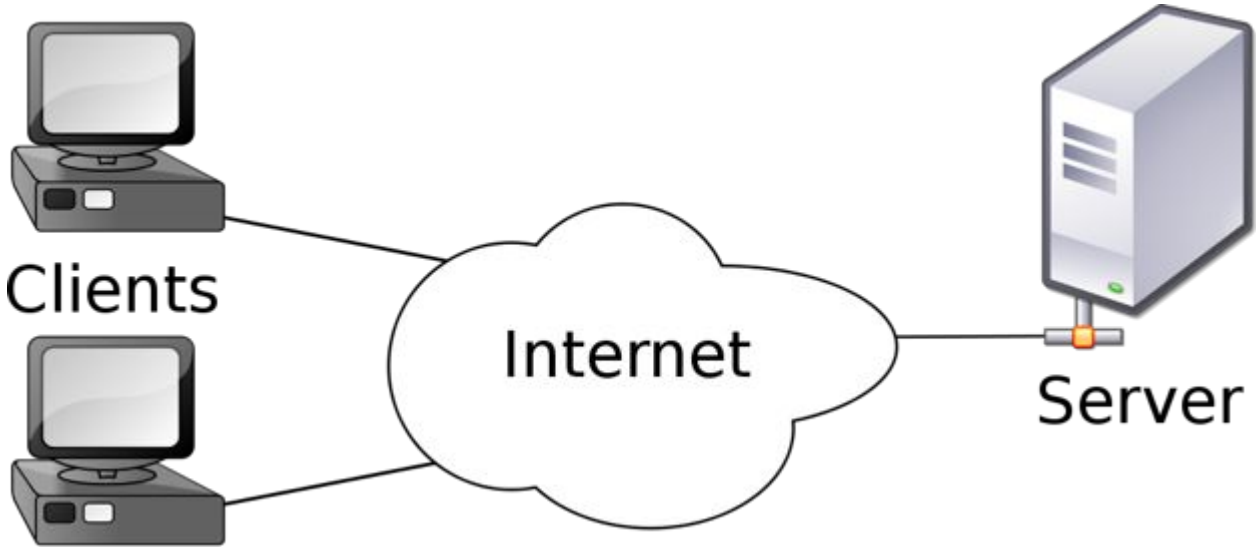
JavaScript History

- It was developed around 1995 when Netscape founder, Marc Andreessen, realized that HTML needed a partner language for rich web content.
- He enlisted Brendan Eich from Santa Clara University to develop the prototype.
- He finished the prototype in exactly 10 days.

The Evolution of JavaScript

- In the late 90s JavaScript was strictly for client side scripting
  - Editing HTML
  - Adding animations and font-types
- JavaScript now has robust frameworks and runtimes to build entire web-applications
  - Angular.js: Popular front-end development for websites
  - React.js: Newer front-end website development, also mobile applications (Facebook and Instagram are written in React Native).
  - Node.js: Server-side
  - Express.js: Server-side Node framework (we'll touch later in the course)

Client/Server Model

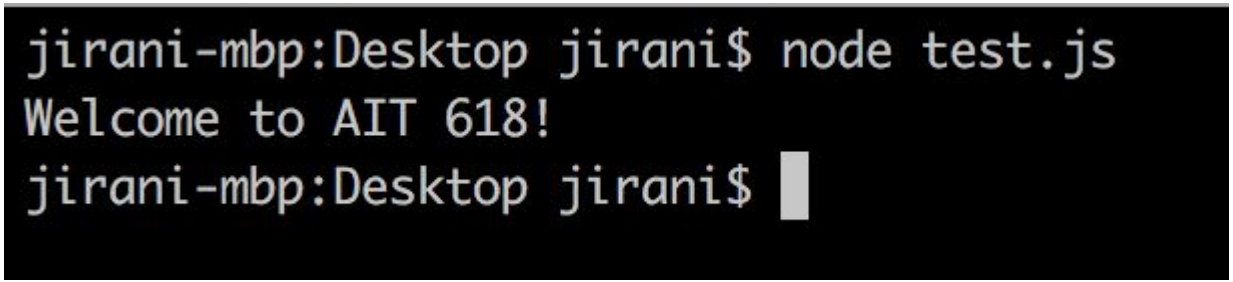


JavaScript Introduction - Print Statement

- If you have familiarity with other programming languages, it is very common to print variables or words to the console for debugging purposes or just for smaller programs.
  - Python uses print
  - Java uses System.out.print
- JavaScript uses console.log();

JavaScript Introduction - Print Statement (cont.)

- Example: If I wrote the line console.log(“Welcome to AIT 618”);
- Output: Welcome to AIT 618

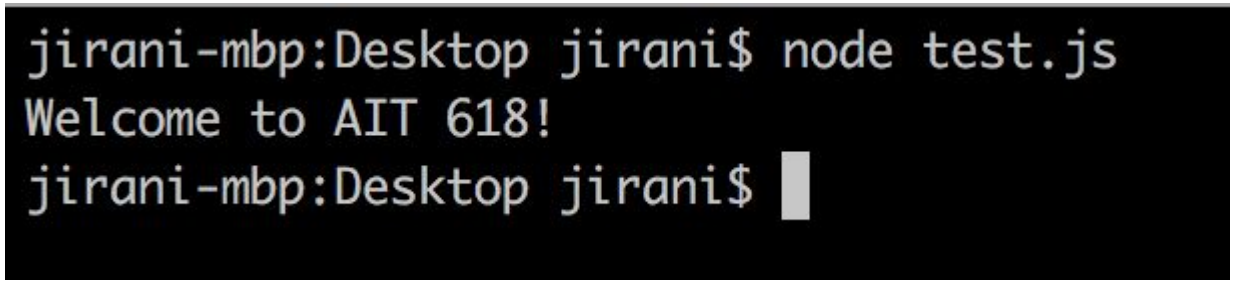


Node.js - Introduction

- Download Node.js here: https://nodejs.org/en/download/
  - For Windows, this tutorial might help: http://blog.teamtreehouse.com/install-node-js-npm-windows
- For my class examples, I use Node.js with the command-line.
  - There are only a few commands to start off that are of importance, so don’t worry or be overwhelmed by a terminal!

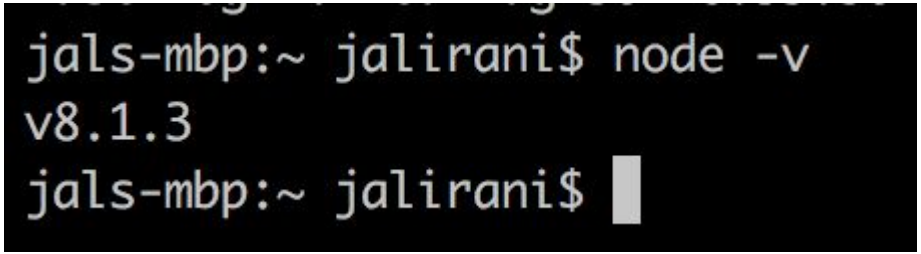
Node.js - Basic Commands

- The “node” command is simply to run a JavaScript file. It is as simple as it sounds.
  - Looking at a previous example, if I have a file called “test.js” and I run “node test.js” I receive an output. See screenshot below:



Node.js - Node Version

- The command “node -v” lists the current version of Node.js that is installed on your machine.

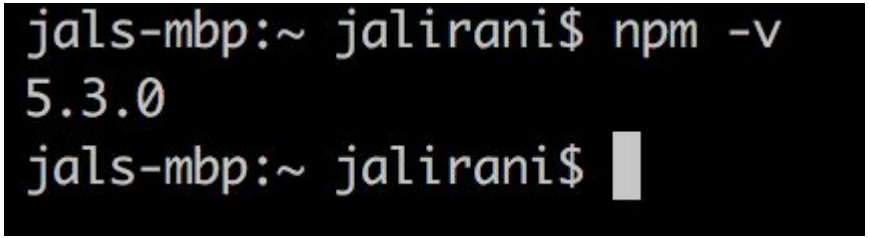


Node.js - npm

- Npm stands for node package manager
- It comes built in when Node.js is downloaded so no need to worry about doing any extra work
- It is a package manager that helps with package installation and different node modules
    - We will cover all of this later in the course, for now just know it helps manage different Node.js libraries to help/assist with project development

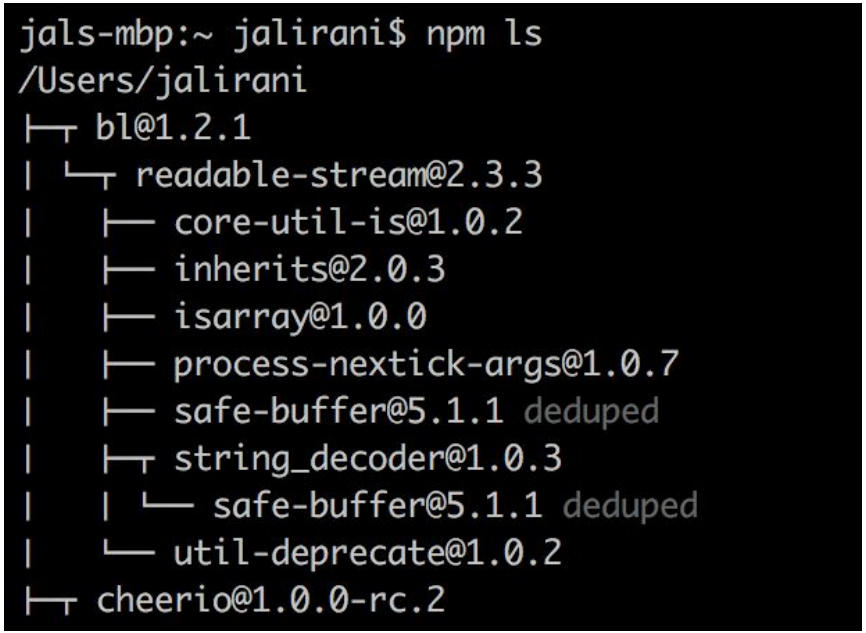
Node.js - npm Version

- Use the command “npm -v” to see what version of Node Package Manager is installed on your machine



Node.js - npm List

- Use the command “npm ls” to see all of the packages that are installed on your machine



Software Stacks

- It is very common in modern Software Engineering or any company that has a tech product to refer to the combination of the subsystems as a “software stack”
- Example: Facebook’s software stack uses Apache, PHP, and a mixture of HTML and React.js front-end
- This course will mainly focus on the MEAN software stack which is currently the most popular.

Software Stacks (cont.)

- M: Mongo Database for data persistence
- E: Express.js for a back-end framework
- R: React.js for front-end design
- N: Node.js for server-side code

Prepare to be a full-stack developer!